PoC on Steganographic File Integrity Checker

(made under Digisuraksha Parhari Foundation)

What is Steganography?

- **Steganography** is the practice of hiding information inside another file (called a *cover file*) in such a way that it doesn't raise suspicion.
- Unlike **cryptography** (which scrambles information so it can't be understood), steganography hides the *existence* of information.
- Example:
    - Hiding text inside an image by tweaking the least significant bits (LSB) of pixel values.
    - To the human eye, the image looks the same, but the hidden data can be extracted with the right program.

Analogy: It's like writing a secret message with invisible ink on a postcard. The postcard looks normal, but someone with UV light can read the hidden message.

This PoC presents a CLI-based Python tool named **StegaFIC** (Steganographic File Integrity Checker). It verifies file integrity by **hashing target files (SHA-256/SHA-3)** and **hiding those hashes inside benign cover files** (PNG images or WAV audio) using **LSB steganography**. Later, the embedded hash is extracted and compared with a freshly computed hash to detect tampering.

While steganography is sometimes used for covert data exfiltration, this tool is built **purely for integrity verification, training, and secure operational workflows**.

Tool: StegaFIC

1. History

The idea of **steganography**—concealing data in plain sight—dates back centuries (invisible inks, microdots). In the digital era, common techniques include manipulating the **Least**

Significant Bits (LSB) of pixels (images) or samples (audio) to hide data without perceptible changes.

StegaFIC (2025) brings steganography together with modern **cryptographic hashing (SHA-256/SHA-3)** to provide a stealthy, portable **file-integrity check** for enterprise, forensic, and training use.

2. Description

StegaFIC generates a file's cryptographic hash and **embeds it into a cover file**. Later, the hidden hash can be extracted and compared against the current hash of the file to detect **any modification**.

Modes:

1. **Embed Mode** – Hash one or more target files and embed the hashes into a chosen **PNG** or **WAV** cover.

2. **Extract Mode** – Extract the hidden hash payload from a cover file.

3. **Verify Mode** – One-shot extraction + comparison with the current target file(s); outputs **intact / modified** result.

4. **Batch Mode** (optional) – Process multiple file–cover pairs from a manifest.

3. What Is This Tool About?

A **stealth integrity checker**.

Instead of shipping a visible ".sha256" file (which can be swapped or blocked by policy), you ship an **ordinary-looking image or audio** that silently carries the reference hash. The receiver uses **StegaFIC** to extract the hidden hash and verify the file's integrity.

Security notes:

- Integrity ≠ confidentiality. We **hide** the hash (don't encrypt the file).
- (Optional) Add **HMAC** with a shared secret to prevent attacker from replacing both file and hidden hash (swap attack).
- Prefer **lossless** covers (**PNG**, **WAV**)—**avoid JPEG/MP3** for embedding (lossy recompression destroys payload).

4. Key Characteristics / Features

- 🔐 **Cryptographic hashing**: SHA-256 / SHA3-256 (selectable).
- 🖼️ **Image (PNG)** or 🔊 **Audio (WAV)** LSB embedding/extraction.
- 🗜️ **Self-describing payload**: algorithm, filename, hash, timestamp, (optional) HMAC.
- 🧪 **Verify**: compares extracted vs current hash → Intact/Modified result.
- 📦 **Batch/manifest** processing (optional).
- 🧰 **Pure CLI**, cross-platform, Python 3.10+.
- 🧱 **Unit tests** for hash, embed/extract, verify.
- 📝 **Audit log** (optional): JSONL of operations & outcomes.

5. Types / Modules Available

- **hashing.py** – streaming file hashing (SHA-256/SHA3-256).
- **stego_png.py** – LSB embedding/extraction for PNG (RGB/A).
- **stego_wav.py** – LSB embedding/extraction for PCM WAV (8/16-bit). (Optional)
- **payload.py** – compact, versioned payload (CBOR/JSON) + optional **HMAC-SHA256**.
- **cli.py** – argparse-based CLI (**embed**, **extract**, **verify**, **batch**).
- **test_stegafic.py** – sures that embedding → extracting → verifying works correctly.
- **demo_run.py** – Example script to run the tool end-to-end without typing long CLI commands.

6. How Will This Tool Help?

- **Enterprise integrity**: Ship reports/contracts with a benign “brand image” carrying the hash.
- **Forensics**: Preserve a hidden chain-of-custody reference for evidence files.
- **Healthcare/Legal**: Discreet integrity markers for sensitive documents.
- **Software distribution**: Provide a logo.png with embedded installer hash.
- **Training**: Hands-on with hashing, stego, and verification workflows.

7. Proof of Concept (PoC) Images

- **Screenshot 1**: **embed** run: **target** → **cover.png** → **success + capacity stats**.

```
(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py --help
usage: stegafic [-h] {embed,extract,verify} ...

positional arguments:
  {embed,extract,verify}

options:
  -h, --help            show this help message and exit

(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py embed -i cover.png -s secret.txt -o stego.png
usage: stegafic embed [-h] --target TARGET --cover COVER --out OUT [--algo ALGO] [--hmac HMAC]
stegafic embed: error: the following arguments are required: --target, --cover, --out

(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py embed --target report.pdf --cover cover.png --out stego.png --algo sha256
Traceback (most recent call last):
  File "D:\Kaam ki cheezen\Python Programs\cli.py", line 63, in <module>
    main()
  File "D:\Kaam ki cheezen\Python Programs\cli.py", line 53, in main
    cmd_embed(args)
  File "D:\Kaam ki cheezen\Python Programs\cli.py", line 12, in cmd_embed
    embed_payload(args.cover, args.out, p)
  File "D:\Kaam ki cheezen\Python Programs\stego_png.py", line 41, in embed_payload
    img = Image.open(cover_path).convert("RGB")
          ^^^^^^^^^^^^^^^^^^^^^
  File "C:\Windows\System32\.venv\Lib\site-packages\PIL\Image.py", line 3513, in open
    fp = builtins.open(filename, "rb")
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'cover.png'

(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py embed --target report.pdf --cover cover.jpeg --out stego.png --algo sha256
Embedded 149 bytes into stego.png (cover cover.jpeg).
```

- **Screenshot 2**: **verify** run: **extraction** + **Intact result**.

```
Administrator: Command Prompt
(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py extract --cover stego.png
Extracted payload bytes length: 149
{"algo":"sha256","filename":"report.pdf","hash":"b78449d72baafd9e9012d36b1f2fec9827de6f5fd62bf6b18ec3081703a58f56","ts":"2025-09-04T19:35:13Z","v":1}
```

```
Administrator: Command Prompt
(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py verify --target report.pdf --cover stego.png
Payload filename: report.pdf
Payload hash  : b78449d72baafd9e9012d36b1f2fec9827de6f5fd62bf6b18ec3081703a58f56
Current hash  : b78449d72baafd9e9012d36b1f2fec9827de6f5fd62bf6b18ec3081703a58f56
[OK] Intact
```

- **Screenshot 3**: **verify** after tampering: **Modified** result (diff hashes shown).

```
Administrator: Command Prompt
(.venv) D:\Kaam ki cheezen\Python Programs>python cli.py verify --target report.pdf --cover stego.png
Payload filename: report.pdf
Payload hash  : b78449d72baafd9e9012d36b1f2fec9827de6f5fd62bf6b18ec3081703a58f56
Current hash  : 9fdbf2c298c2323c8970b025f2fa0f0e294da004fb94d73f6293087132a37c83
[ALERT] Modified
```

- **Screenshot 4**: All required files

1. **hashing.py**

```python
# hashing.py
import hashlib
from typing import Callable

ALGOS = {
    "sha256": lambda: hashlib.sha256(),
    "sha3_256": lambda: hashlib.sha3_256()
}

def get_file_hash(path: str, algo: str = "sha256", block_size: int = 65536) -> str:
    """
    Stream file and return hex digest.
    """
    algo = algo.lower()
    if algo not in ALGOS:
        raise ValueError(f"Unsupported algorithm: {algo}")
    h = ALGOS[algo]()
    with open(path, "rb") as f:
        while True:
            chunk = f.read(block_size)
            if not chunk:
                break
            h.update(chunk)
    return h.hexdigest()

if __name__ == "__main__":
    import argparse
    p = argparse.ArgumentParser()
    p.add_argument("file")
    p.add_argument("--algo", default="sha256")
    args = p.parse_args()
    print(get_file_hash(args.file, args.algo))
```

2. payload.py

hashing.py | payload.py* | stego_png.py | cli.py | demo_run.py | test_stegafic.py

```python
# payload.py
import json
import time
import hmac
import hashlib
from typing import Optional, Dict

PAYLOAD_VERSION = 1

def make_payload(filename: str, hash_hex: str, algo: str = "sha256",
                 hmac_key: Optional[bytes] = None) -> bytes:
    """
    Create a JSON payload (bytes) that can be embedded.
    If hmac_key provided, attach an HMAC for authenticity.
    """
    payload = {
        "v": PAYLOAD_VERSION,
        "algo": algo,
        "filename": filename,
        "hash": hash_hex,
        "ts": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime())
    }
    payload_json = json.dumps(payload, separators=(",", ":"), sort_keys=True)
    if hmac_key:
        mac = hmac.new(hmac_key, payload_json.encode("utf-8"), hashlib.sha256).hexdigest()
        payload["hmac"] = mac
        payload_json = json.dumps(payload, separators=(",", ":"), sort_keys=True)
    return payload_json.encode("utf-8")

def parse_payload(payload_bytes: bytes, hmac_key: Optional[bytes] = None) -> Dict:
    """
    Parse and validate payload bytes. If hmac_key provided, verify presence & validity.
    Returns payload dict (raises on HMAC mismatch).
    """
    payload_json = payload_bytes.decode("utf-8")
    payload = json.loads(payload_json)
    if hmac_key:
        if "hmac" not in payload:
            raise ValueError("Payload missing HMAC but key provided for verification.")
        mac = payload.pop("hmac")
        recomputed = hmac.new(hmac_key, json.dumps(payload, separators=(",", ":"), sort_keys=True).encode("utf-8"), hashlib.sha256).hexdigest()
        if not hmac.compare_digest(mac, recomputed):
            raise ValueError("HMAC mismatch - payload tampered or wrong key.")
        payload["hmac"] = mac
    return payload
```

3. stego_png.py

hashing.py | payload.py | stego_png.py | cli.py | demo_run.py | test_stegafic.py

```python
# stego_png.py
from PIL import Image
import math
from typing import Tuple

MAGIC = b"STG1"  # 4-byte magic to find payload start

def _bytes_to_bits(b: bytes):
    for byte in b:
        for i in range(8):
            yield (byte >> (7 - i)) & 1

def _bits_to_bytes(bits):
    out = bytearray()
    cur = 0
    cnt = 0
    for bit in bits:
        cur = (cur << 1) | bit
        cnt += 1
        if cnt == 8:
            out.append(cur)
            cur = 0
            cnt = 0
    return bytes(out)

def capacity_in_bytes(img: Image.Image) -> int:
    """
    Calculate how many payload BYTES can be stored in this image using 1 LSB per channel (R,G,B).
    We'll use 3 bits per pixel (RGB). Returns floor(bytes).
    """
    w, h = img.size
    channels = 3  # using RGB channels' LSBs
    total_bits = w * h * channels
    return total_bits // 8
```

hashing.py | payload.py | stego_png.py | cli.py | demo_run.py | test_stegafic.py

```python

def embed_payload(cover_path: str, out_path: str, payload: bytes) -> None:
    """
    Embed payload (bytes) into PNG cover (lossless) using 1 LSB per R/G/B channel.
    Payload format saved: MAGIC + 4-byte length (big-endian) + payload
    """
    img = Image.open(cover_path).convert("RGB")
    cap = capacity_in_bytes(img)
    full = MAGIC + len(payload).to_bytes(4, "big") + payload
    if len(full) > cap:
        raise ValueError(f"Payload too large ({len(full)} bytes) for cover (capacity {cap} bytes).")
    pixels = list(img.getdata())
    bits = _bytes_to_bits(full)
    new_pixels = []
    bit_iter = iter(bits)
    for px in pixels:
        r, g, b = px
        try:
            r = (r & ~1) | next(bit_iter)
        except StopIteration:
            new_pixels.append((r, g, b))
            continue
        try:
            g = (g & ~1) | next(bit_iter)
        except StopIteration:
            new_pixels.append((r, g, b))
            continue
        try:
            b = (b & ~1) | next(bit_iter)
        except StopIteration:
            new_pixels.append((r, g, b))
            continue
        new_pixels.append((r, g, b))
    # If some pixels left untouched (bit_iter exhausted), append them unchanged
    if len(new_pixels) < len(pixels):
        new_pixels.extend(pixels[len(new_pixels):])
    out_img = Image.new("RGB", img.size)
    out_img.putdata(new_pixels)
    out_img.save(out_path, "PNG")
    print(f"Embedded {len(payload)} bytes into {out_path} (cover {cover_path}).")
```

```python
def extract_payload(stego_path: str) -> bytes:
    """
    Extract payload from stego PNG. Returns payload bytes.
    """
    img = Image.open(stego_path).convert("RGB")
    pixels = list(img.getdata())
    bits = []
    for r, g, b in pixels:
        bits.append(r & 1)
        bits.append(g & 1)
        bits.append(b & 1)
    # Convert first bytes to find MAGIC and length
    first_bytes = _bits_to_bytes(bits[:8* (4 + 4)])  # enough to read magic + length (approx)
    if not first_bytes.startswith(MAGIC):
        # Try searching for magic in bitstream sliding window
        all_bytes = _bits_to_bytes(bits)
        idx = all_bytes.find(MAGIC)
        if idx == -1:
            raise ValueError("No payload magic found in image.")
        # read length
        length = int.from_bytes(all_bytes[idx+4:idx+8], "big")
        payload = all_bytes[idx+8: idx+8+length]
        return payload
    else:
        # simple case: magic appears at start
        magic = first_bytes[:4]
        length = int.from_bytes(first_bytes[4:8], "big")
        total_needed_bits = (4 + 4 + length) * 8
        total_bits = bits[:total_needed_bits]
        all_bytes = _bits_to_bytes(total_bits)
        payload = all_bytes[8:8+length]
        return payload

```

4. cli.py

```python
# cli.py
import argparse
import sys
from hashing import get_file_hash
from payload import make_payload, parse_payload
from stego_png import embed_payload, extract_payload

def cmd_embed(args):
    h = get_file_hash(args.target, algo=args.algo)
    key = args.hmac.encode("utf-8") if args.hmac else None
    p = make_payload(args.target, h, algo=args.algo, hmac_key=key)
    embed_payload(args.cover, args.out, p)

def cmd_extract(args):
    pbytes = extract_payload(args.cover)
    print("Extracted payload bytes length:", len(pbytes))
    print(pbytes.decode("utf-8"))

def cmd_verify(args):
    pbytes = extract_payload(args.cover)
    key = args.hmac.encode("utf-8") if args.hmac else None
    payload = parse_payload(pbytes, hmac_key=key)
    target_hash = get_file_hash(args.target, algo=payload["algo"])
    print("Payload filename:", payload.get("filename"))
    print("Payload hash  :", payload.get("hash"))
    print("Current hash  :", target_hash)
    if payload.get("hash") == target_hash:
        print("[OK] Intact")
    else:
        print("[ALERT] Modified")

def main():
    p = argparse.ArgumentParser(prog="stegafic")
    sub = p.add_subparsers(dest="cmd")

    se = sub.add_parser("embed")
    se.add_argument("--target", required=True)
    se.add_argument("--cover", required=True)
    se.add_argument("--out", required=True)
    se.add_argument("--algo", default="sha256")
    se.add_argument("--hmac", default=None, help="Optional HMAC secret")

    sx = sub.add_parser("extract")
    sx.add_argument("--cover", required=True)

    sv = sub.add_parser("verify")
    sv.add_argument("--target", required=True)
    sv.add_argument("--cover", required=True)
    sv.add_argument("--hmac", default=None)

    args = p.parse_args()
    if args.cmd == "embed":
        cmd_embed(args)
    elif args.cmd == "extract":
        cmd_extract(args)
    elif args.cmd == "verify":
        cmd_verify(args)
    else:
        p.print_help()
        sys.exit(1)

if __name__ == "__main__":
    main()

```

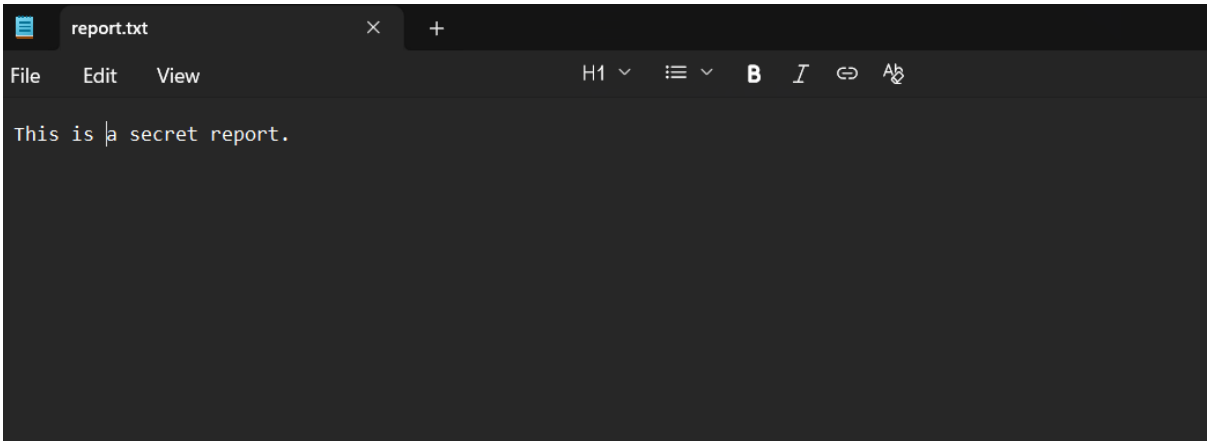
5. demo_run.py

```python
# demo_run.py
import os, base64, subprocess, sys
from hashing import get_file_hash

COVER_B64 = """iVBORw0KGgoAAAANSUhEUgAAAEAAAAAQCAYAAAB49mG8AAAACXBIWXMAAAsTAAALEwEAmpwYAAAB
SULEQVR4nO2WUQ7DIAxFv3cE9yA3sA3sQ3sA3nYg7sA7sQ3uDq0aHh2GhCzE0gQ1kG/A4s4n2s5k
7s8D1q2n8gSxwA9e6y0qk5g2m7k0g8g4h3rV+c0R7pJXx9qI1UJ0KJM6p1kQo0w0h6Qe8kG2Qe8k
G2Qe8kG2Qe8kG2Qe8kG2Qe8kG2Qe8kG2Qe8kG2Qe8kE2w/H5kQ3T6s3Hfgn8v4Y
ZfV0q7QIDAQAB
"""
with open("cover_sample.png", "wb") as f:
    f.write(base64.b64decode(COVER_B64))

with open("target.txt", "w") as f:
    f.write("This is the target file for StegaFIC demo.\n")

print("Target file hash:", get_file_hash("target.txt"))
print("Embedding...")
os.system('python cli.py embed --target target.txt --cover cover_sample.png --out stego_cover.png --algo sha256')
print("Extraction & verify:")
os.system('python cli.py verify --target target.txt --cover stego_cover.png')
print("Now tamper the target and verify again (simulates modified file).")
with open("target.txt", "a") as f:
    f.write("tamper\n")
print("New target hash:", get_file_hash("target.txt"))
os.system('python cli.py verify --target target.txt --cover stego_cover.png')
```

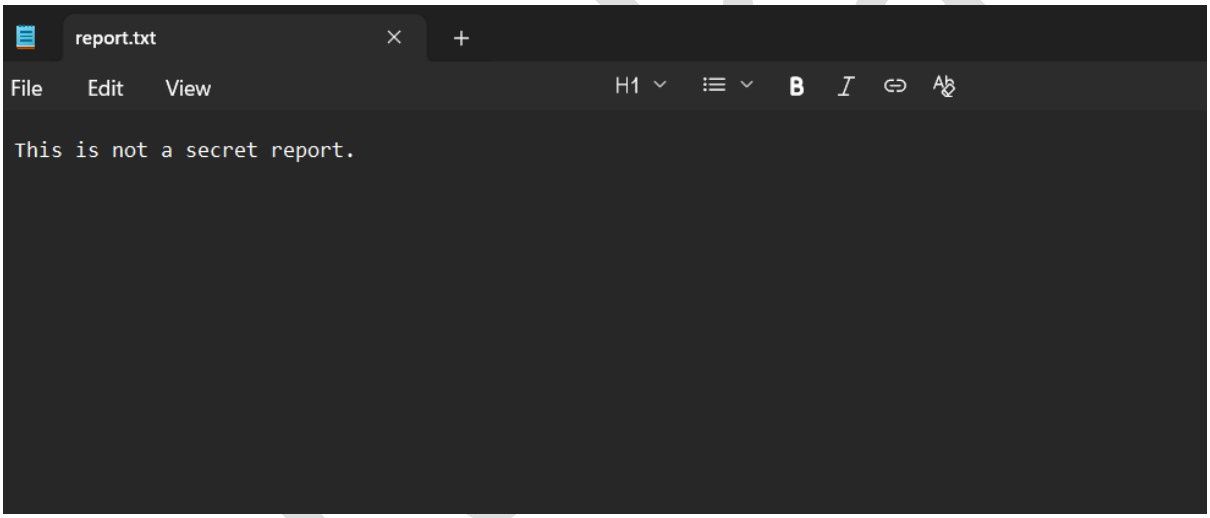
6. test_stegafic.py

```python
# test_stegafic.py
import os
from hashing import get_file_hash
from payload import make_payload, parse_payload
from stego_png import embed_payload, extract_payload

def test_hash_and_payload(tmp_path):
    target = tmp_path / "t.txt"
    target.write_text("hello")
    h = get_file_hash(str(target))
    p = make_payload("t.txt", h)
    # use provided small cover (create simple RGB)
    from PIL import Image
    cover = tmp_path / "c.png"
    img = Image.new("RGB", (64,64), color=(100,150,200))
    img.save(str(cover))
    out = tmp_path / "s.png"
    embed_payload(str(cover), str(out), p)
    extracted = extract_payload(str(out))
    parsed = parse_payload(extracted)
    assert parsed["hash"] == h
```

7. Original report.pdf/txt file (target file):



8. **Modified report.pdf/txt file:**



9. **Another random image file downloaded for cover, cover.jpeg (where the hash of target file is embedded)**

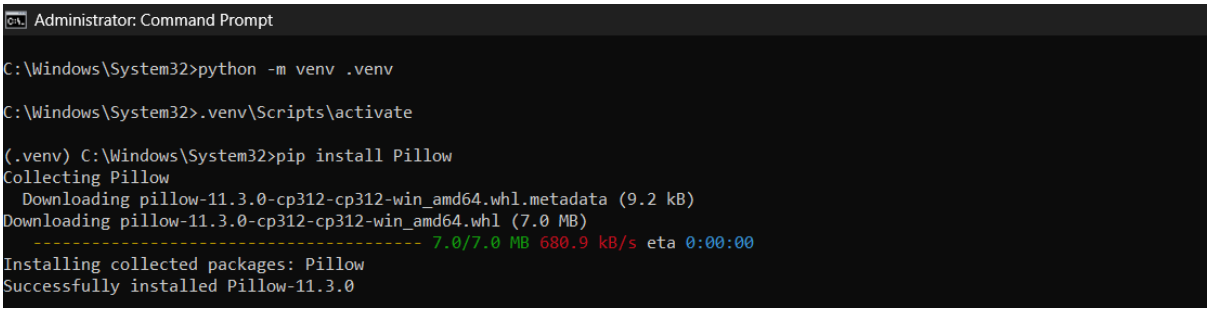
8. How to Set Up & Run

Requirements

- Python **3.10+**
- Libraries: **Pillow**, **numpy** (for PNG path), or standard **wave** for WAV; **hashlib**, **hmac**, **argparse**.

Installation

- python -m venv .venv && source .venv/bin/activate # (Windows: .venv\Scripts\activate)
- pip install pillow numpy



9. Usage Scenarios & Examples

Example 1: Enterprise Document

- Alice hashes **q2_financials.pdf** and embeds into **brand_banner.png**.
- Bob receives both, runs **verify** → **Intact**.
- If HR re-exports the PDF after edits, **verify** → **Modified**.

Example 2: Forensic Evidence

- Analyst embeds evidence hash into **scene_photo.png** at seizure time.
- Any later manipulation triggers **Modified** at testimony time.

Example 3: Software Release

- Publish **product_logo.png** carrying the installer's SHA-256.
- Users run **verify** before installation to confirm integrity.

10. 15-Liner Summary

- Stealth **file-integrity** via steganography
- **SHA-256/SHA3-256** hashing
- **PNG/WAV** LSB embedding
- **CLI** and cross-platform
- Optional **HMAC** against swap attacks
- Self-describing, versioned payload
- Batch processing support
- Capacity checks & warnings
- Clear **Intact/Modified** verdicts
- Streamed hashing (large files OK)
- Minimal dependencies
- Unit-tested core functions
- JSONL audit logging (optional)

- Suitable for enterprise/forensics/training
- Demo shows tamper detection

11. Time to Use / Best Case Scenarios

- Shipping sensitive docs with stealth integrity metadata
- Evidence handling and chain-of-custody workflows
- Software distribution & patch verification
- Security awareness and classroom labs

12. When to Use During Investigation

- To **prove non-tampering** of digital artifacts
- To **reproduce** or **demonstrate** hidden-hash integrity checks
- To **document** integrity states at multiple custody points (timestamps)

13. Best Person to Use & Required Skills

Best Users

- DFIR / Forensic analysts
- Enterprise IT / SecOps
- Release engineering / Compliance teams

Skills

- Basic terminal fluency
- Understanding of hashing & integrity
- Awareness of steganography caveats (lossy formats, recompression risks)

14. Flaws / Suggestions to Improve

- **JPEG/MP3** not supported for embedding (lossy) → add robust watermarking research mode.
- **HMAC requires key management** → add keyring/OS-secret store integration.
- **Capacity constraints** (small covers) → auto-select cover or **multi-cover sharding**.
- No **public-key signatures** → add **Ed25519** signing option alongside hash.
- No **tamper-evident transparency log** → integrate **Sigstore/Rekor** style append-only log.

- Optional **GUI** (Tkinter/PyQt) for non-technical users.

15. Good About the Tool

- **Harmless & training-friendly**
- **Stealthy** integrity metadata (no obvious **.sha256** files)
- **Simple CLI**, minimal deps
- **Deterministic** & fast
- **Extensible** (HMAC, batch, logs)
- **Portable** across OSes

🌍 Real-World Scenarios

1. Secure Document Verification in Enterprises

- Companies often share confidential reports (like financial data or contracts).
- Instead of sending a separate checksum file (which could be tampered with), the hash is hidden inside a cover image/audio (like the company logo or a sound clip).
- When the document is received, the hidden hash is extracted and compared to verify integrity.

👉 **Benefit**: The hash is stealthy (not obvious) and adds a hidden layer of integrity checking.

---

2. Forensic Evidence Handling

- Law enforcement agencies capture digital evidence (images, videos, reports).
- To ensure these files aren't altered in transit or storage, their hashes can be hidden inside other innocuous-looking files.
- Later, in court, the hashes can be extracted to prove the evidence hasn't been tampered with.

👉 **Benefit**: Hidden integrity verification ensures chain-of-custody without exposing hash files.

---

3. Medical Records Protection

- Hospitals exchange sensitive medical records like scans and lab reports.
- A patient's MRI scan file hash could be hidden inside another scan or even an audio note, ensuring that the medical data wasn't modified by unauthorized parties.

👉 **Benefit**: Confidential health data remains trustworthy.

---

4. Software Distribution / Updates

- When software updates are released, instead of providing a separate checksum (which hackers might replace), the hash could be embedded into a cover image (like the product logo) shipped along with the installer.
- Users extract it to verify the installer is authentic.

👉 **Benefit:** Prevents checksum spoofing and makes tampering harder to detect.

---

5. Military or Intelligence Communication

- Sensitive intelligence reports could have their integrity hashes hidden inside everyday images/audio shared over insecure channels.
- Even if intercepted, outsiders just see a normal-looking image, while insiders can extract the hidden hash to verify the real file.

👉 **Benefit**: Stealth + security in high-risk environments.

Some Key Points:

- **Why We Created a Virtual Environment**

We made a virtual environment (.venv) for a few important reasons:

1. **Isolation –** Keeps all dependencies (like Pillow) separate from your system Python so they don't conflict with other projects.
2. **Reproducibility –** Anyone else running this project can recreate the same environment (same Python version + same libraries).
3. **Clean Workspace –** No global pollution; when we're done, we can just delete the .venv folder and the system stays untouched.
4. **Professional Practice –** In cybersecurity / Python projects, virtual envs are the standard way to package and ship PoCs safely.

- **Functions of All the Files**

1. **hashing.py**
    - Handles cryptographic hashing (SHA256, SHA1, MD5, etc.).
    - Core job: take a file (like report.pdf) and generate its hash digest for embedding.
2. **payload.py**
    - Prepares the payload (the hash + metadata) that will be hidden inside the image.
    - Can also extract payload later for verification.
3. **stego_png.py**
    - Core steganography engine.
    - Embeds the payload inside a PNG (cover file).
    - Also handles extraction of hidden payloads from a stego file.
4. **cli.py**
    - The command-line interface.
    - Exposes commands like embed, extract, verify.
    - Handles user input and calls the correct functions from the above modules.

5. **demo_run.py**
    - Example script to run the tool end-to-end without typing long CLI commands.
    - Useful for quick testing and screenshots.
6. **test_stegafic.py**
    - Unit testing script.
    - Ensures that embedding → extracting → verifying works correctly.
    - Good for debugging if something breaks.

- Made by
  Sarthaka Subhankara
  Singh
  Intern ID – 438
  Digisuraksha Parhari
  Foundation